

Custom APEX error handling function

November 30, 2021

W

When your app is ready for users, you want to mask (trap/catch, translate, log) error

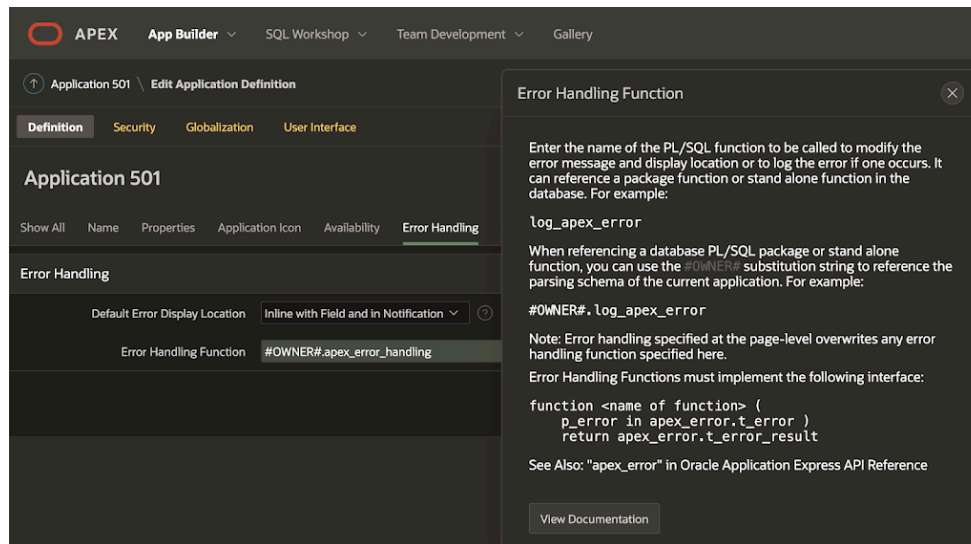
[SHARE](#)

messages which slipped through your standard PL/SQL error handler and show user something less revealing. You might also want to store these errors in your own error log table and not in the APEX log.

To achieve this you need to define an Error Handling Function in Shared Components, Application Definition Attributes, Error Handling. Use your own name, mine is "apex_error_handling".

Labels

apex



To create this function you can get inspired like me by the nice example in Oracle documentation, in [APEX_ERROR](#) package, section [Example of an Error Handling Function](#). I modified this function a bit to serve my needs:

- show whole error message with backtrace to APEX developers
- show just a error type and log_id to common users
- show constraint names on integrity errors (you can easily convert them to relevant table and columns)
- log everything (specially the component causing the error and the error backtrace) to my table
- let the application exception through (for me SQLCODE=-20000)

Main goal of this function is to transform record
APEX_ERROR.T_ERROR (p_error) on the input to
record APEX_ERROR.T_ERROR_RESULT. You can find
records description in section **Constants and
Attributes** except the t_error.component record.

Structures follows:

```
type t_error is record (  
    message                varchar2(32767),    -- Error message which  
    additional_info        varchar2(32767),    -- Only used for displ  
    display_location       varchar2(40),        -- Use constants "used  
    association_type       varchar2(40),        -- Use constants "used  
    page_item_name        varchar2(255),        -- Associated page ite  
    region_id              number,              -- Associated tabular  
    column_alias           varchar2(255),        -- Associated tabular  
    row_num                pls_integer,        -- Associated tabular  
    apex_error_code        varchar2(255),        -- Contains the system  
    is_internal_error      boolean,             -- Set to TRUE if it's  
    is_common_runtime_error boolean,            -- TRUE for internal a  
    ora_sqlcode            number,              -- SQLCODE on exceptio  
    ora_sqlerrm            varchar2(32767),    -- SQLERRM which trigg
```

```

error_backtrace      varchar2(32767),  -- Output of sys.dbms_
error_statement      varchar2(32767),  -- Statement that was
component            apex.t_component  -- Component which has

```

```

type t_error_result is record (
    message          varchar2(32767),  -- Error message which will be
    additional_info  varchar2(32767),  -- Only used for display_locat
    display_location varchar2(40),      -- Use constants "used for dis
    page_item_name   varchar2(255),    -- Associated page item name
    column_alias     varchar2(255)     -- Associated tabular form col

```



The hidden definition:

```

type t_component is record (
    type varchar2(30),      -- APEX dictionary view name of the compon
    id   number,            -- Internal component id which triggered t
    name varchar2(32767)    -- Name of the component which triggered t

```



The function

Now you are ready to create your function:

```
CREATE OR REPLACE FUNCTION apex_error_handling (  
    p_error          APEX_ERROR.T_ERROR  
)  
  
RETURN APEX_ERROR.T_ERROR_RESULT  
AS  
  
    out_result      APEX_ERROR.T_ERROR_RESULT;  
  
    --  
  
    l_log_id        NUMBER;                -- log_id from your lo  
    l_log_name       VARCHAR2(64);         -- short error type vi  
    l_component      VARCHAR2(64);         -- to identify source  
    --  
  
    app_exception    CONSTANT NUMBER      := -20000; -- your ap  
  
BEGIN  
  
    out_result := APEX_ERROR.INIT_ERROR_RESULT(p_error => p_error);  
  
  
    -- assign log_id sequence (app specific, probably from sequence)  
    IF p_error.ora_sqlcode IN (-1, -2091, -2290, -2291, -2292) THEN  
        -- handle constraint violations  
        -- ORA-00001: unique constraint violated  
        -- ORA-02091: transaction rolled back (can hide a deferred con  
        -- ORA-02290: check constraint violated  
        -- ORA-02291: integrity constraint violated - parent key not f  
        -- ORA-02292: integrity constraint violated - child record fou  
        l_log_name := 'CONSTRAINT_ERROR' || ' ' || APEX_ERROR.EXTRACT_  
            p_error          => p_error,
```

```

        p_include_schema    => FALSE
    );
    --
    out_result.message        := l_log_name;
    out_result.display_location := APEX_ERROR.C_INLINE_IN_NOTIFICA
    --
ELSIF p_error.is_internal_error THEN
    l_log_name := 'INTERNAL_ERROR';
ELSE
    l_log_name := 'UNKNOWN_ERROR';
END IF;

-- store incident in your log
l_component := TO_CHAR(APEX_APPLICATION.G_FLOW_STEP_ID) || ' ' ||
--
l_log_id := log_error (
    l_log_name,
    l_component,
    p_error.ora_sqlerrm,
    p_error.error_statement,
    p_error.error_backtrace
);

-- mark associated page item (when possible)
IF out_result.page_item_name IS NULL AND out_result.column_alias I
    APEX_ERROR.AUTO_SET_ASSOCIATED_ITEM (
        p_error          => p_error,
        p_error_result   => out_result
    )

```

```

    );
END IF;

-- show only the latest error message to common users
IF (is_developer() OR p_error.ora_sqlcode = app_exception) THEN
    out_result.message := l_log_name || '|' || TO_CHAR(l_log_id) |
        l_component || '<br />' ||
        out_result.message || '<br />' ||
        APEX_ERROR.GET_FIRST_ORA_ERROR_TEXT(p_error => p_error);
    out_result.additional_info := '';
ELSE
    out_result.message      := l_log_name || '|' || TO_CHAR(l_
    out_result.additional_info := '';
END IF;
--
RETURN out_result;
END;
/

```

Here is a helping function to recognize if the current user is developer or not. You can remove it if you don't need it.


```

CREATE OR REPLACE FUNCTION is_developer (
    in_username          VARCHAR2          := NULL
)
RETURN BOOLEAN AS
    is_valid             VARCHAR2(1);
BEGIN
    SELECT 'Y' INTO is_valid
    FROM apex_workspace_developers d
    JOIN apex_applications a
        ON a.workspace          = d.workspace_name
    WHERE a.application_id      = sess.get_app_id()
        AND d.is_application_developer = 'Yes'
        AND d.account_locked     = 'No'
        AND COALESCE(in_username, sess.get_user_id()) IN (UPPER(d.user
    GROUP BY d.email;
    --
    RETURN (is_valid = 'Y');
END;
/

```

Here is a mockup of the custom log function.

```

CREATE OR REPLACE FUNCTION log_error (
    in_log_name      logs.log_name%TYPE,
    in_component     logs.component%TYPE,
    in_message       logs.message%TYPE      := NULL,
    in_statement     logs.statement%TYPE    := NULL,
    in_backtrace     logs.backtrace%TYPE    := NULL
)
RETURN logs.log_id%TYPE
AS
    PRAGMA AUTONOMOUS_TRANSACTION;
    --
    rec          logs%ROWTYPE;
BEGIN
    rec.log_id      := log_id.NEXTVAL;      -- your sequence
    rec.log_name    := in_log_name;
    rec.component   := in_component;
    rec.message     := in_message;
    rec.statement   := in_statement;
    rec.backtrace   := in_backtrace;
    --
    INSERT INTO log_errors VALUES rec;
    COMMIT;
    --
    RETURN rec.log_id;
EXCEPTION
WHEN OTHERS THEN
    ROLLBACK;

```

```
        RAISE_APPLICATION_ERROR(-20000, 'LOG_FAILED', TRUE);  
    END;  
/
```

Modify it however you like and don't forget to put these functions into a package.

TAGS: [APEX](#)



[SHARE](#)

Comments

Anonymous · September 19, 2023 at 12:53 AM

Thank you.it is very clear.

why do you need the function called log_error ?what info

does it adding to the developer ?

all the info already exists in the table

APEX_DEBUG_MESSAGES

Anonymous · September 19, 2023 at 5:56 AM

It is my personal preference. You might log using APEX feature, you might log through Logger or into custom table. If you wrap this in the log_error function, you can switch the target anytime without changing the whole application.

REPLY



Sasha Gomanuke · May 11, 2025 at 5:08 AM

This comment has been removed by the author.

REPLY



Sasha Gomanuke · May 11, 2025 at 5:08 AM

This comment has been removed by the author.

REPLY



Sasha Gomanuke · May 11, 2025 at 5:09 AM

Just try to call PL-SQL from APEX callback that is calling from ajax apex.server.process Apex API. Global function doesn't catch it...

REPLY



Enter Comment





Jan Květina

Get in touch with me at

www.jankvetina.cz

[VISIT PROFILE](#)

Tags



Archive



 Powered by Blogger